ModelOptimizationandTuningPhaseTemplate

Date	22April2026
Name	Omkar Bhimrao Magdum
ProjectTitle	RestaurantRecommendation system
MaximumMarks	10 Marks

ModelOptimizationandTuningPhase

The Model Optimization and Tuning Phase involves improving our machine learning recommendationmodeltogetthebestperformance.Thisincludesadjustingthemodel'sparameters, experimenting with different algorithms, and selecting the most suitable model based on evaluation metrics such as accuracy, precision, recall, and RMSE (Root Mean Squared Error).

Ourrestaurantrecommendationsystemwasdesignedtosuggestsimilarrestaurantsbasedon

Model	TunedHyperparameters
Model 1: Content-Based Filtering	- **SimilarityMetric:**Cosinesimilaritywasusedastheprimarymetricto compute similarity between restaurants based on features like cuisines, rating, and cost. - **TopNRecommendations:**Thenumberoftopsimilarrestaurants returned was tested with values like 5, 10, and 15.

location,userratings,cuisines,andcostusingcollaborativefilteringandcontent-basedfiltering techniques.

HyperparameterTuningDocumentation(8Marks):

```python
def recommend(name, cosine_similarities = cosine_similarities):

    # Create a list to put top restaurants
    recommend_restaurant = []

    # Find the index of the hotel entered
    idx = indices[indices == name].index[0]

    # Find the restaurants with a similar cosine-sim value and order them from bigger number
    score_series = pd.Series(cosine_similarities[idx]).sort_values(ascending=False)

    # Extract top 30 restaurant indexes with a similar cosine-sim value
    top30_indexes = list(score_series.iloc[0:31].index)

    # Names of the top 30 restaurants
    for each in top30_indexes:
        recommend_restaurant.append(list(df_percent.index)[each])

    # Creating the new data set to show similar restaurants
    df_new = pd.DataFrame(columns=['cuisines', 'Mean Rating', 'cost'])

    # Create the top 30 similar restaurants with some of their columns
    for each in recommend_restaurant:
        df_new = df_new.append(pd.DataFrame(df_percent[['cuisines','Mean Rating', 'cost']][df_percent.index == each].sample()))

    # Drop the same named restaurants and sort only the top 10 by the highest rating
    df_new = df_new.drop_duplicates(subset=['cuisines','Mean Rating', 'cost'], keep='False')
    df_new = df_new.sort_values(by='Mean Rating', ascending='False').head(10)

    print('TOP %s RESTAURANTS LIKE %s WITH SIMILAR REVIEWS: ' % (str(len(df_new)), name))
    df_new.index = df_new.index.str.lower()
    return df_new
```

Model 2:

Collaborative

Filtering

- **Algorithm:**SVD(SingularValueDecomposition)fromtheSurprise library.
- **LearningRate:**Tunedvaluessuchas0.005,0.01,and0.02weretested.
- **Regularization:**Parameterssuchas0.02,0.05weretriedto avoid overfitting.
- **NumberofEpochs:**Adjusted between20and100 epochs.

```python
from surprise import SVD, Dataset, Reader
from surprise.model_selection import cross_validate

reader = Reader(rating_scale=(1, 5))
data = Dataset.load_from_df(df[['user_id', 'restaurant_name', 'rating']], reader)

svd = SVD()
cross_validate(svd, data, measures=['RMSE', 'MAE'], cv=5, verbose=True)
```

FinalModelSelectionJustification(2Marks):

FinalModel	Reasoning
Model1:Content-Based Filtering	Selectedduetoitssimplicityandgoodperformancewithoutrequiring detailed user history. It gave interpretable and relevant results using restaurant features like cuisines, ratings, and cost.